Software Engineering

Dr. Fatma Eskander
Math.department, Faculty of Science, Mansoura University

Use-Case Diagrams Problems

Core Components of a Use-Case Diagram

1. System Boundary:

2. Actor:

3. Use Case:

4. Relationships:

Aspect	Association	Include «include»	Extend «extend»
Line Type	Solid line	Dashed arrow	Dashed arrow
Direction	No specific direction	Base → Included	Extension → Base
Between	Actor ↔ Use Case	Use Case → Use Case	Use Case → Use Case
Execution	Direct interaction	**MANDATORY** - Always executes	**OPTIONAL** - Conditional execution
Purpose	Show who uses the system	Reuse common functionality	Add optional behavior

EX (1)

- A library needs a new software system. Based on the requirements, a use-case diagram was created. The system, named "Library Management System," is represented by a large bounding box. **Members** should be able to search for books, borrow books, and return books. **Librarians** should be able to add new books, update book information, and remove old books. The "Borrow Book" process must follow checking the member's account for any outstanding fines.

EX (1)

- **1. What represents the System Boundary?**

a) The list of all requirements given in the scenario.

b) The large labeled "Library Management System".

c) The network connection between the member and the library.

d) The library building itself.

The system boundary is the box that defines the scope of the system being modeled, containing all the use cases inside it.

EX (1)

- **2. Who are the Actors in this system?**

a) Search Catalog, Borrow Book, Return Book

b) Book, Fine, Catalog

c) Member and Librarian

d) The Library Database

Actors are the external entities that interact with the system.

EX (1)

- **3. Which of the following is a complete and correct list of the Use Cases?**

a) Member, Librarian, Search Catalog, Borrow Book

b) Search Catalog, Borrow Book, Return Book, Add New Book, Update Book Info, Remove Book, Check for Fines

c) Login, Search Catalog, Borrow Book, Add New Book

d) Check for Fines, Process Payment, Generate Report

These are all the specific units of functionality (goals) represented as ovals within the system boundary.

EX (1)

- **4. What is the nature of the relationship between 'Borrow Book' and 'Check for Fines'?**

a) An **Association**, meaning they are loosely connected.

b) An **Extend** relationship <<extend>>, meaning it is optional.

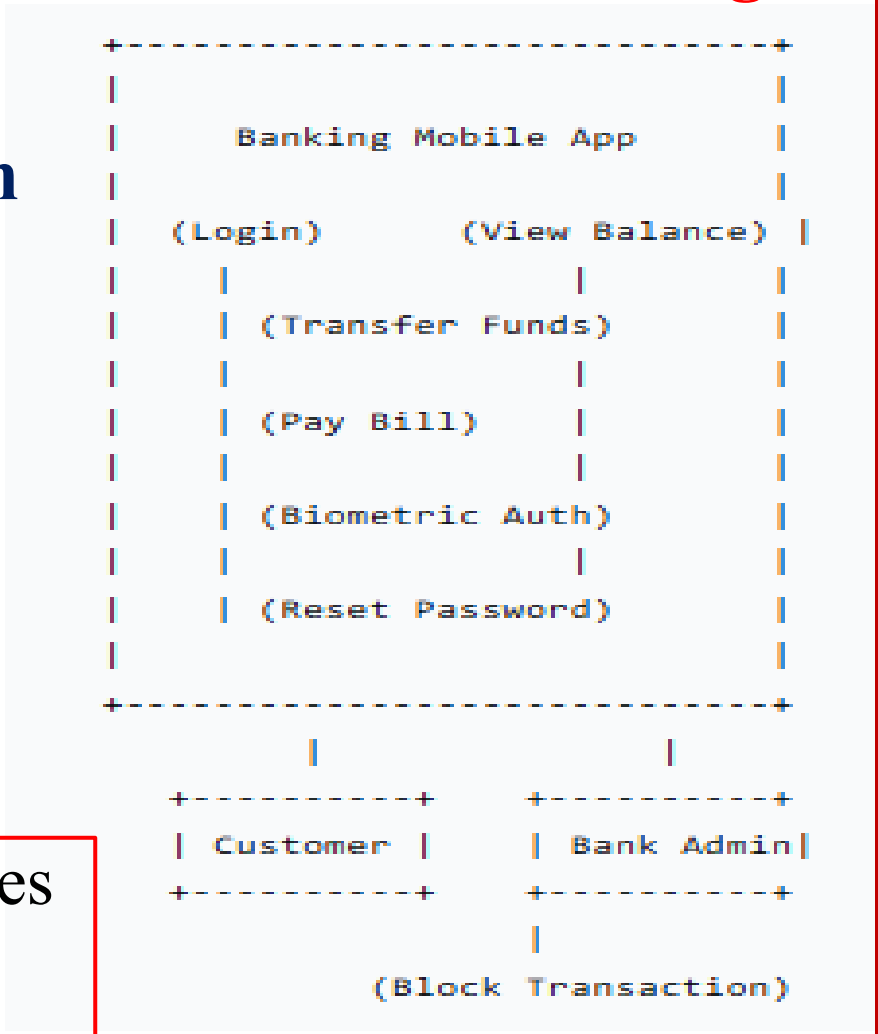
c) An **Include** relationship<<include>>, meaning it is mandatory.

d) None

This means the "Check for Fines" behavior is a mandatory part of completing the "Borrow Book" use case.

EX (2)

- **Based on this use-case diagram, answer the following questions:**



1- What represents the **System Boundary** in this diagram?

a) The mobile phone the customer is holding.

b) The database where transaction records are stored.

c) The box containing all the use cases like "Login" and "Transfer Funds".

d) The Bank Admin's office.

EX (2)

- **Based on this use-case diagram, answer the following questions:**

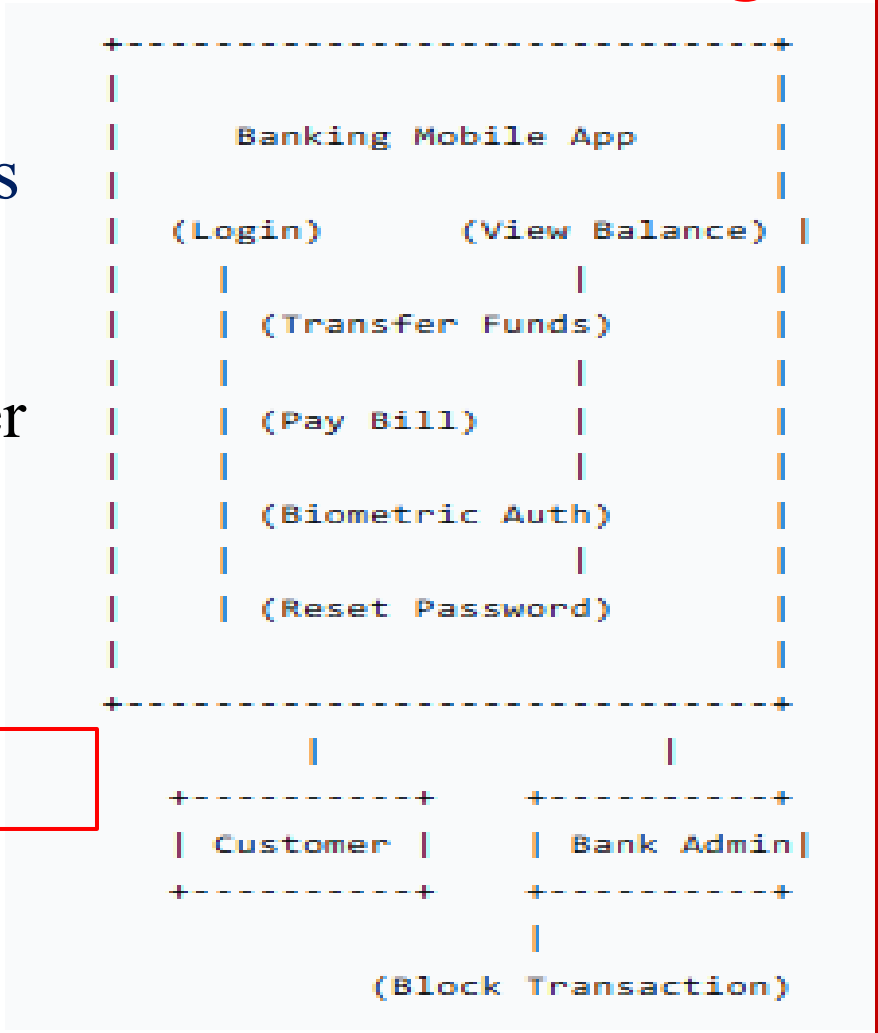
2- Who are the **Actors** in this system?

a) Login, View Balance, and Transfer Funds

b) Mobile Device and Server

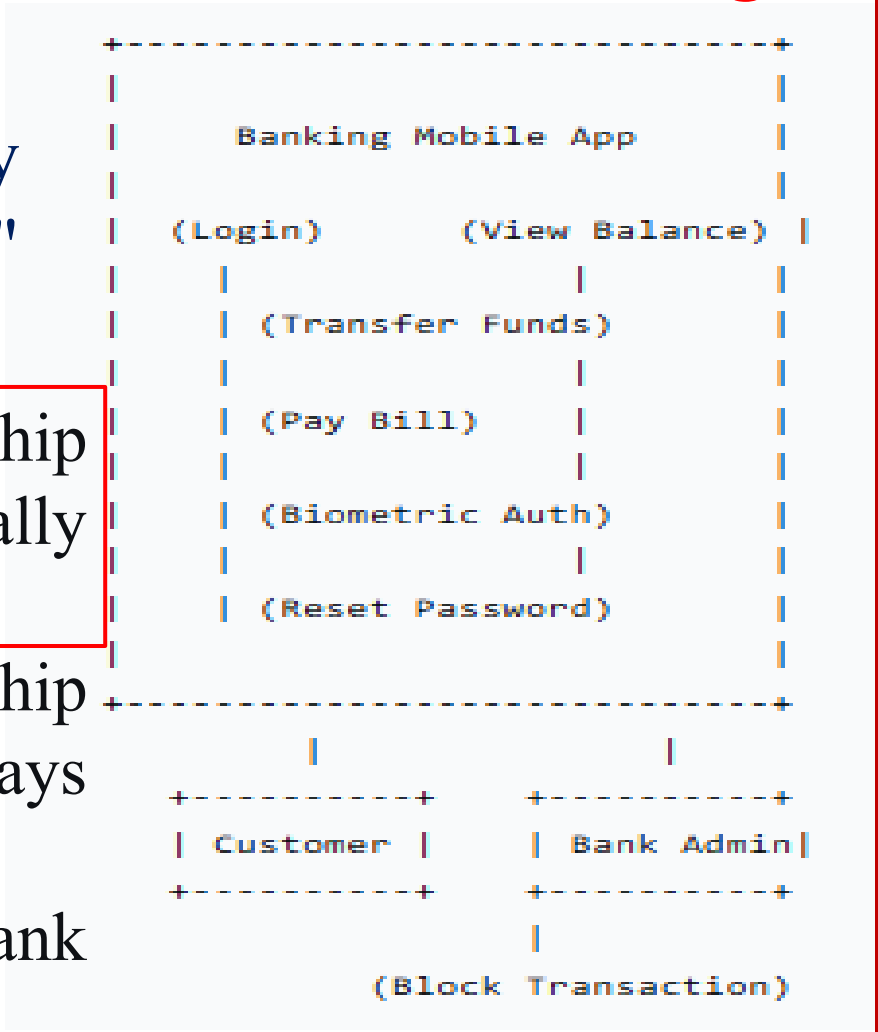
c) Customer and Bank Admin

d) Password and Biometric Data



EX (2)

- **Based on this use-case diagram, answer the following questions:**



3- What is the most likely relationship between "Login" and "Biometric Auth"?

a) An <<extend>> relationship where Biometric Auth may optionally enhance Login.

b) An <<include>> relationship where Login must always execute Biometric Auth.

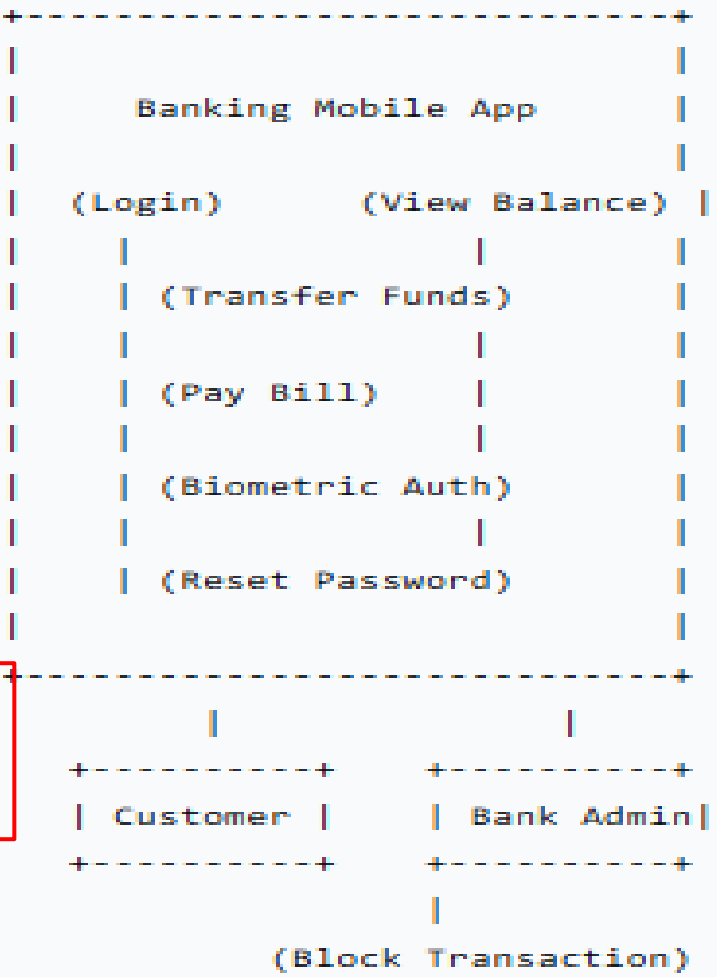
c) An association where Bank Admin initiates Biometric Auth.

d) None

EX (2)

- **Based on this use-case diagram, answer the following questions:**

4- In a standard Use-Case Diagram, what does the direction of the dashed arrow for an <<include>> relationship indicate?



a) It points from the actor to the use case they initiate.

b) It points from the base use case TO the mandatory sub-function use case.

c) It points from the optional use case TO the base use case it extends.

d) It points from the system boundary to the external actor.

Activity Diagrams Problems

Activity diagram

An activity diagram is used to model the **dynamic aspects** of a system. It represents the **flow from one activity to another**.

Element	Meaning	Symbol
Activities	Business processes and tasks	
Start node	Start of process	
End node	End of process	
Transition	move from one activity to another	
Synchronization bar	represents **fork** and **join**. Where **fork** represents the concurrent activities, and **join** means end of concurrent activities and is used to move to next sequence activity after finishing concurrent ones.	
Decision node	represent alternative ways out of an activity. It has 2 outgoing arcs	
Merge node	brings together alternative flow. It has one outgoing flow.	
Guard	represents the condition (Boolean test); Guards are associated with transitions. A transition takes place only if its guard evaluates to true at the time the flow of control reaches that transition.	[*condition*]
Swimlane	Swimlanes group **activities** associated with different **roles**. Each swimlane shows who is responsible for which set of related activities.	

EX (3)

- **Based on the business rules from the previous lecture ("Orders over $100 qualify for free shipping," "Payment must be approved before shipping"), model the "Process Customer Order" workflow.**
- **Create an Activity Diagram for this process.**

Solution:

1- Start Node. 

2- Activity: Receive Order 

3- Activity: Validate Payment

4- Decision Node: [Payment Valid?]

- If No → **Activity:** `Notify Customer of Payment Failure` → **End Node.**
- If Yes → Proceed.

5- Activity: Check Order Total

6- Decision Node: [Order Total > $100?]

- If Yes → **Activity:** `Apply Free Shipping`
- If No → **Activity:** `Calculate Shipping Cost`

7- Merge Node: (Paths from the shipping decision re-join here).

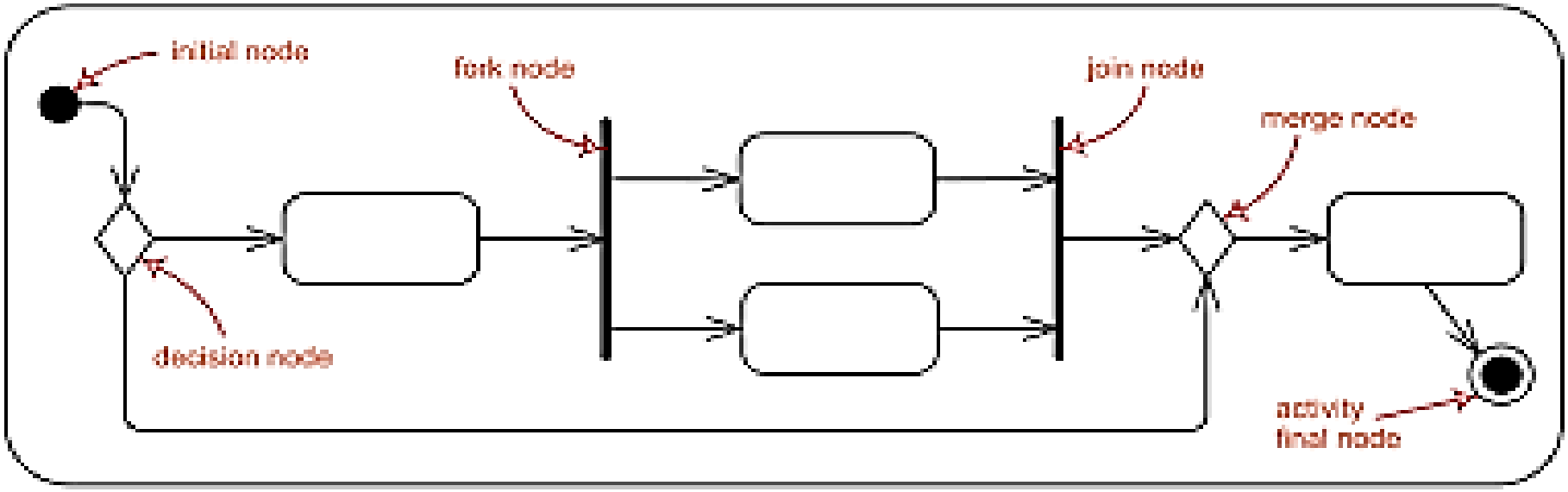
8- Activity: Prepare Order for Shipment

9. Activity: Ship Order

10. End Node.

Note that:

A decision node directs the flow conditionally, while a **merge node** simply rejoins parallel flows after a condition has been met



EX (4): User Login Process Analysis

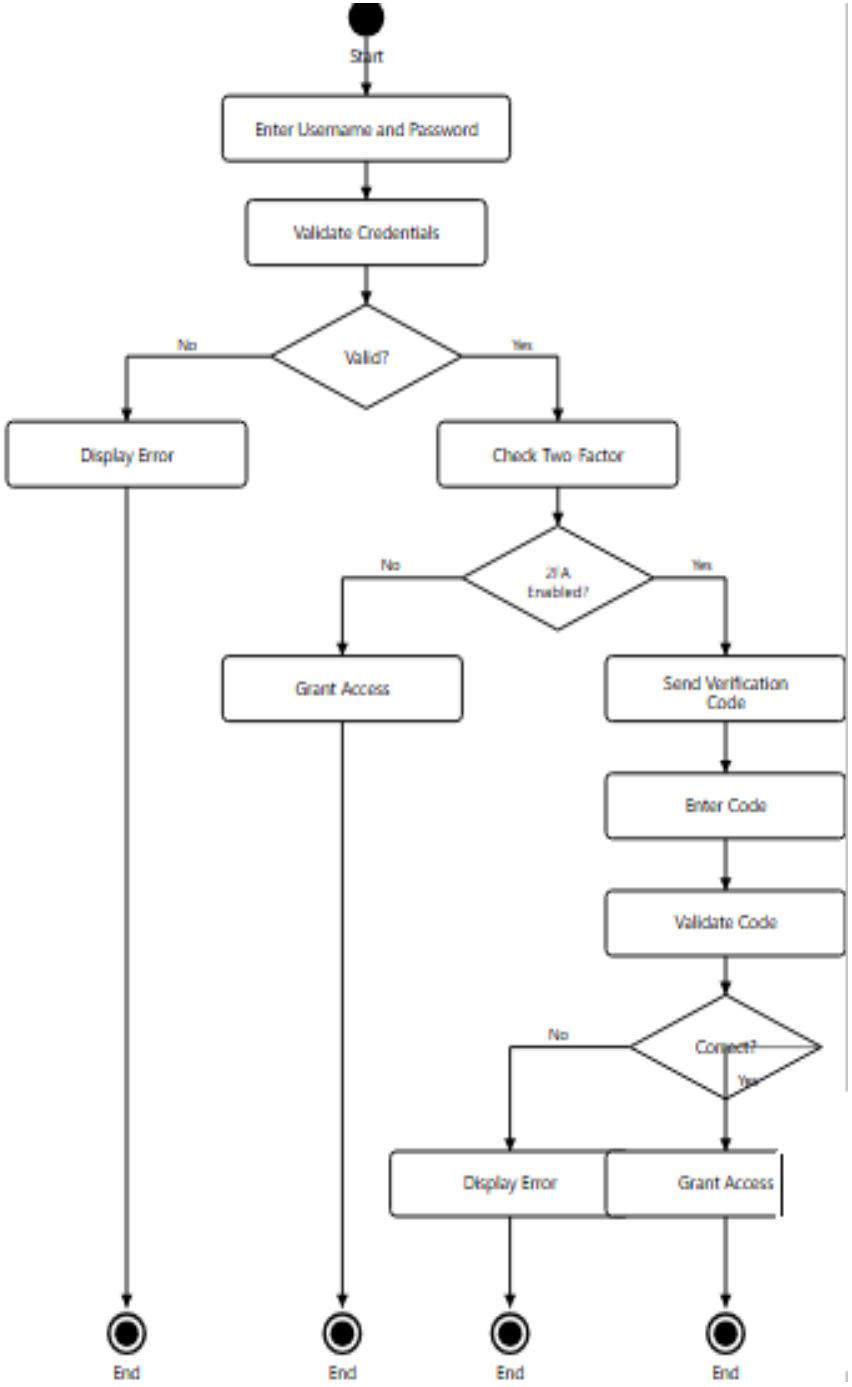
1- What symbol represents the **start** of the activity diagram?

a) Rectangle

b) Diamond

c) Filled black circle (●)

d) Bull's eye circle (◉)



EX (4)

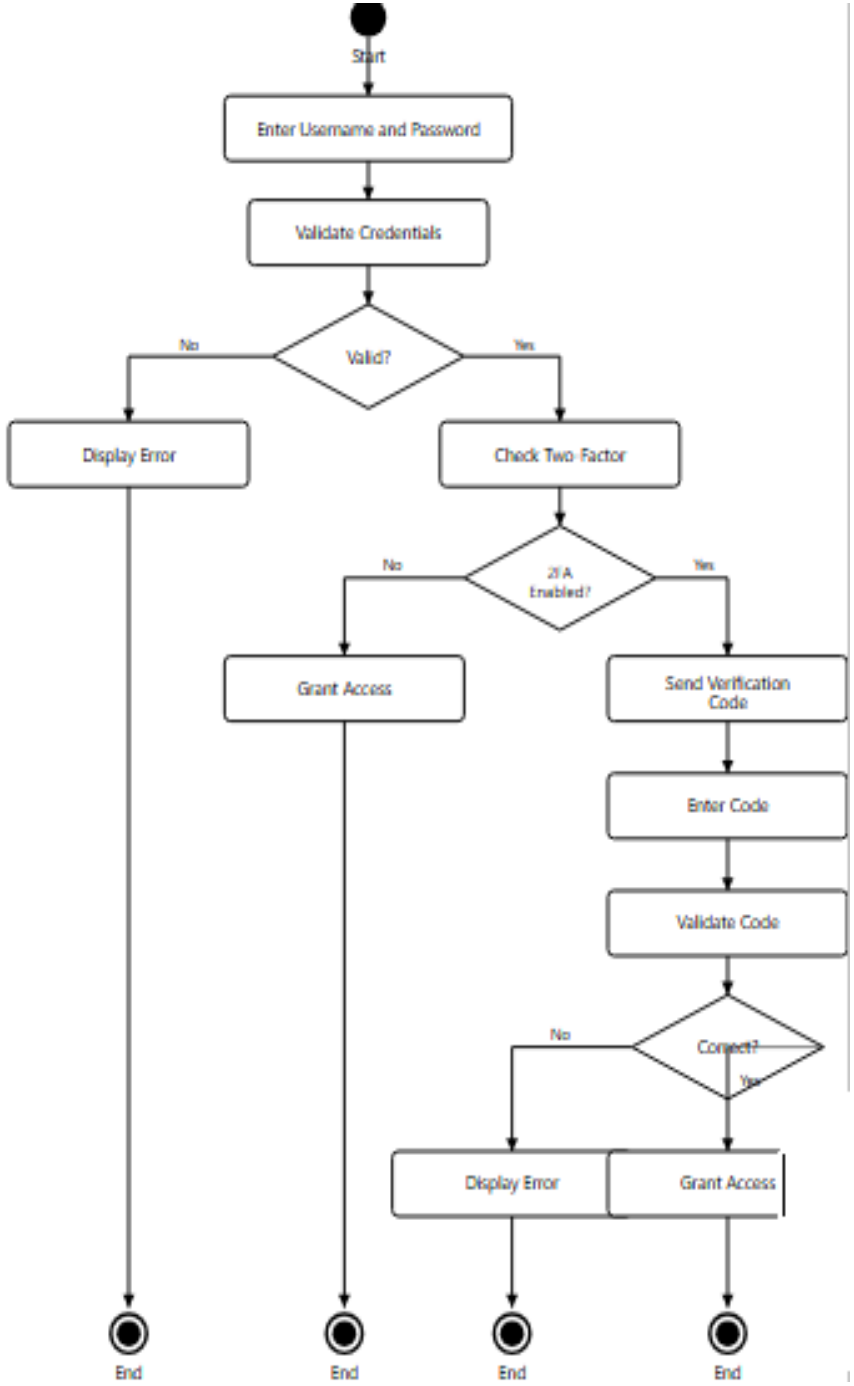
2- How many decision nodes are in this activity diagram?

a) 2

b) 3

c) 4

d) 5



EX (4)

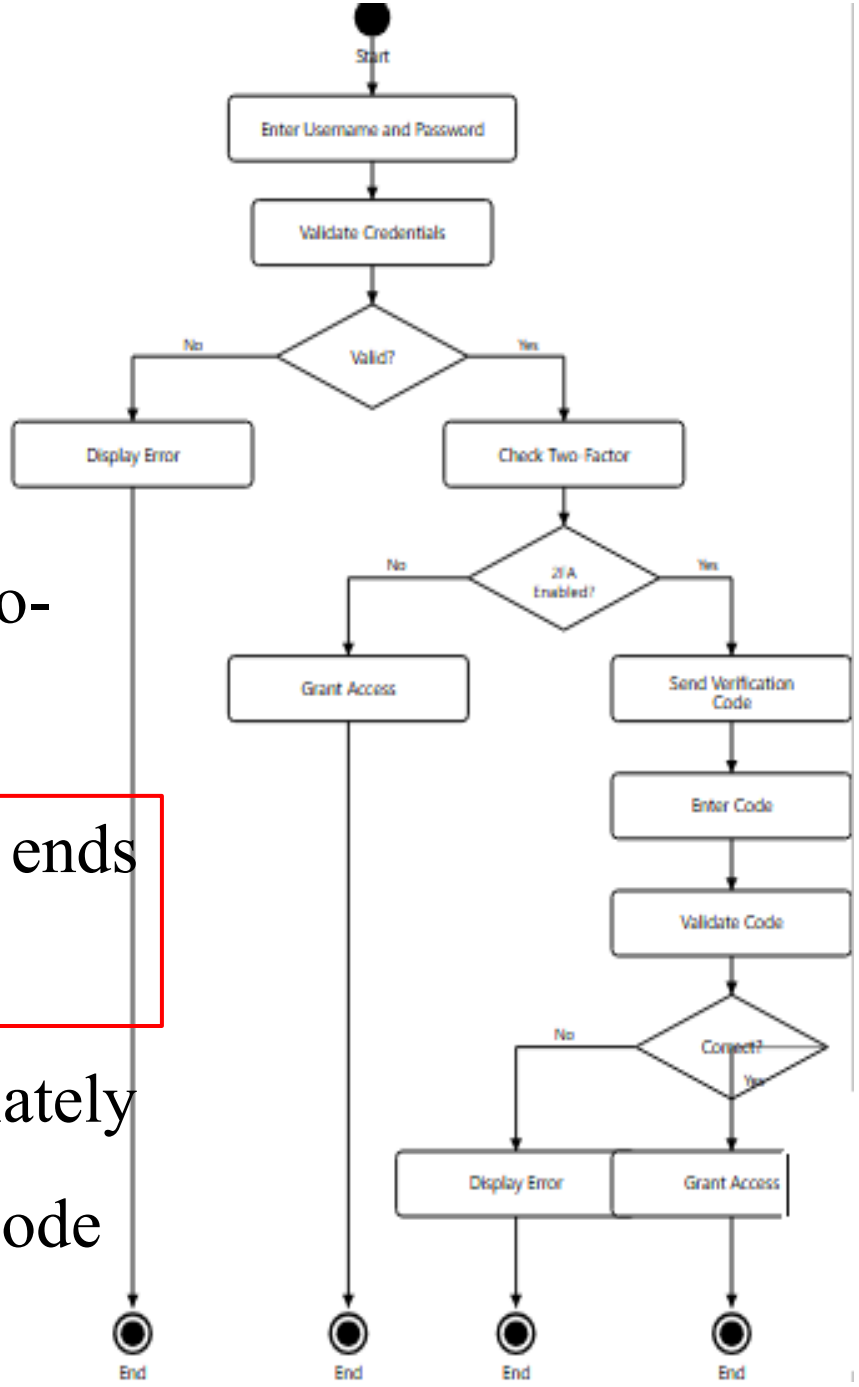
3- What happens after "Validate Credentials" if the credentials are invalid?

a) The system proceeds to check two-factor authentication

b) The system displays an error and ends the process

c) The system grants access immediately

d) The system sends a verification code



EX (4)

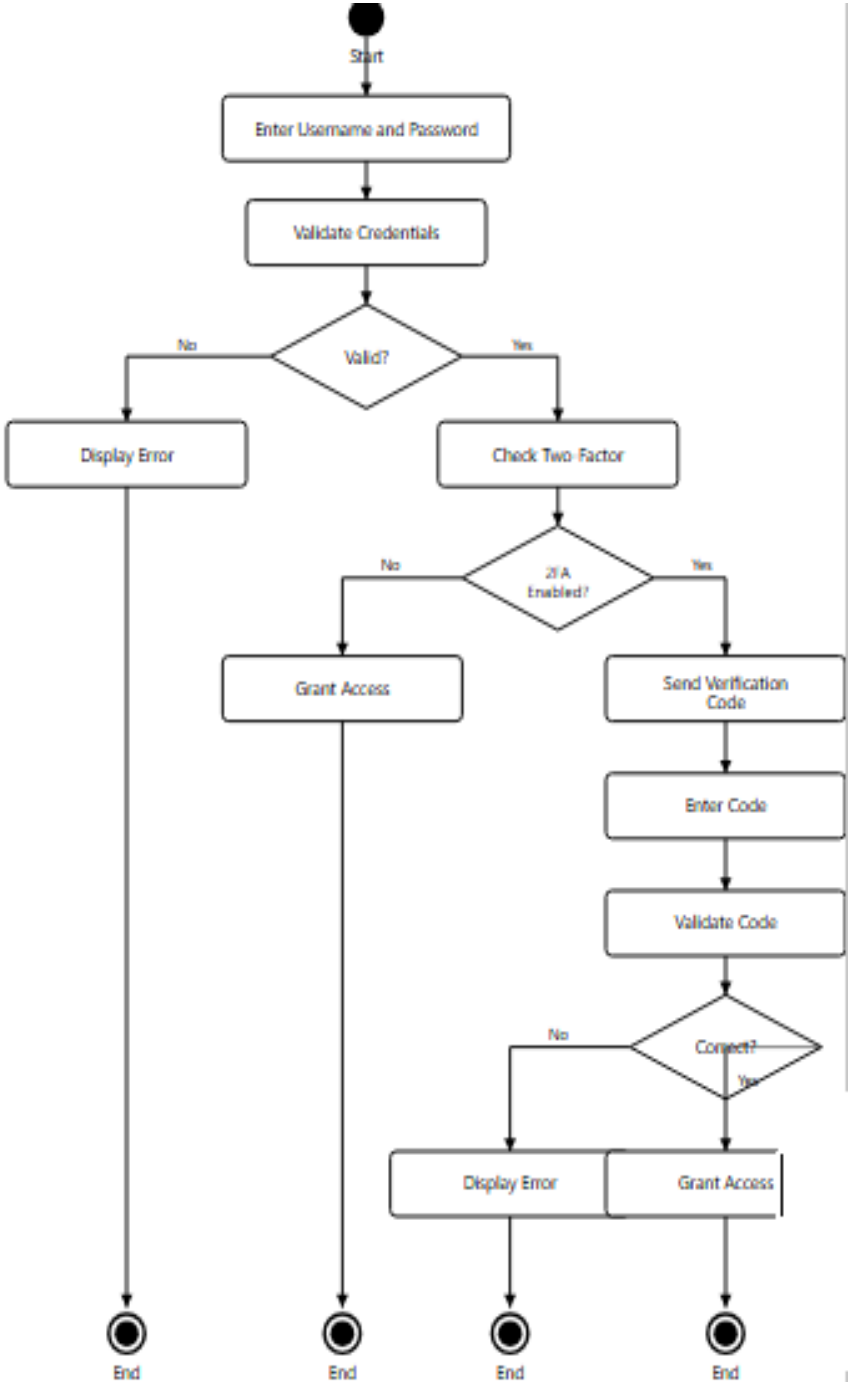
4- If 2FA is NOT enabled, what is the next step after "Check Two-Factor"?

a) Send Verification Code

b) Display Error

c) Grant Access

d) Enter Code



EX (4)

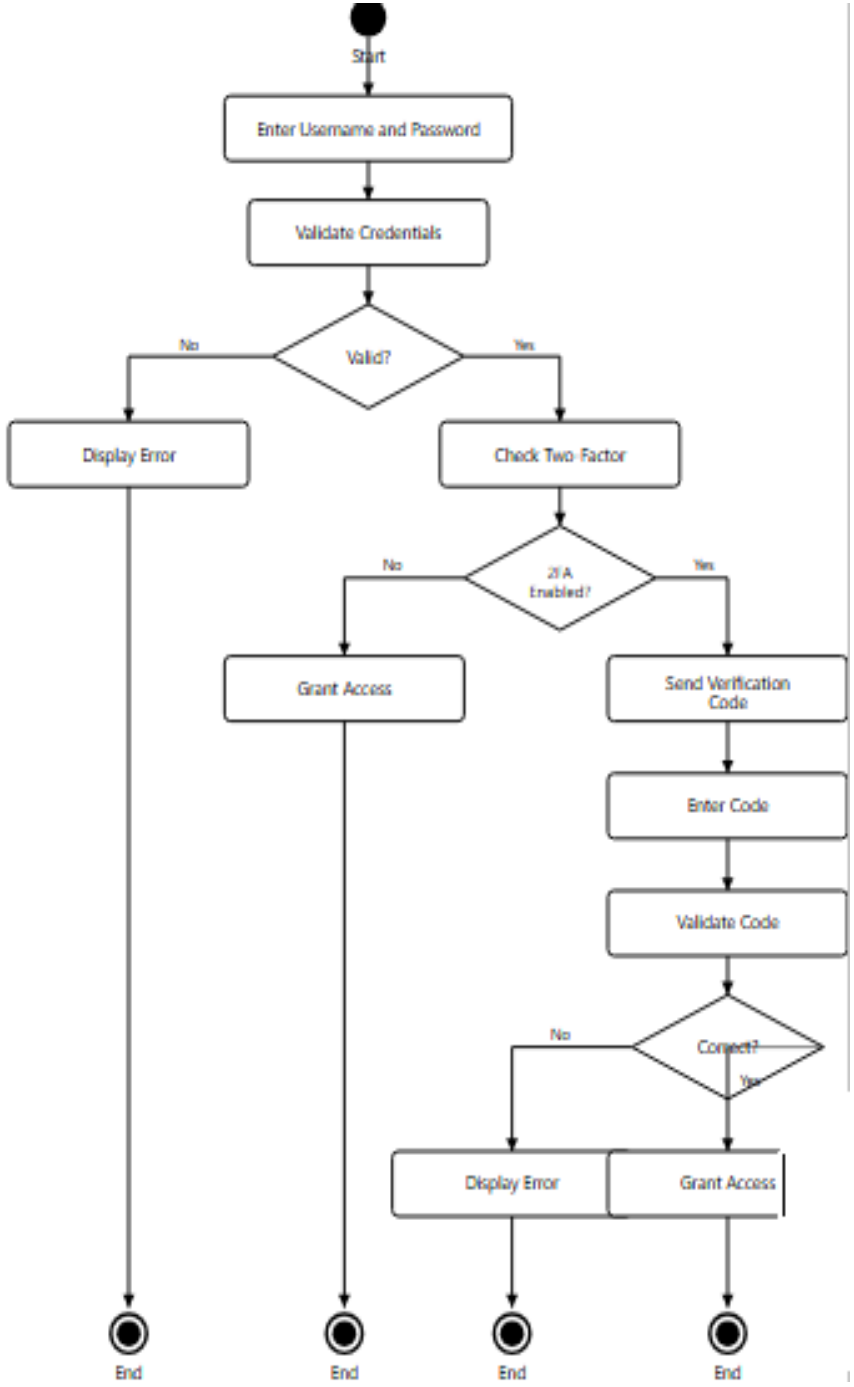
5- How many possible final nodes does this activity diagram have?

a) 1

b) 2

c) 3

d) 4



Introduction to UML, OO modelling and class diagrams

22/11/2025

Modelling

What is Modeling?

A model is an **abstraction** of a system

Abstraction allows us to ignore unessential details

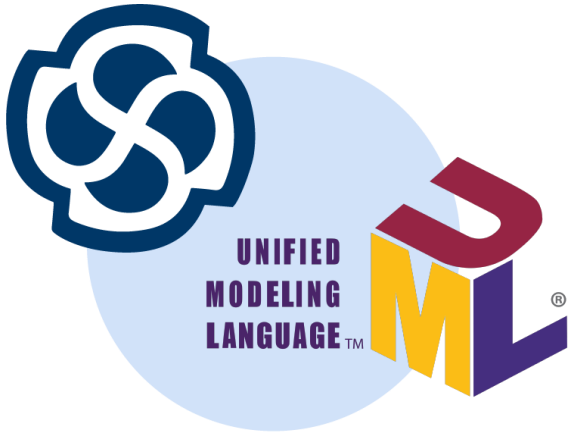
Why building models?

- To reduce complexity
- To test the system before building it.
- To document and visualize your ideas

UML modelling

- **What is UML?**

Unified Modeling Language (UML)



A standardized visual language for modeling software systems.

Used for specifying, visualizing, constructing, and documenting software systems

Types of UML Diagrams

- UML includes 14 diagram types divided into **two categories**:

1- Structural Diagrams (Static View):

- Class Diagram
- Object Diagram
- Component Diagram
- Deployment Diagram
- Package Diagram
- Composite Structure Diagram
- Profile Diagram

Types of UML Diagrams

2- Behavioral Diagrams (Dynamic View):

- Use Case Diagram ✓
- Activity Diagram ✓
- State Machine Diagram
- Sequence Diagram
- Communication Diagram
- Interaction Overview Diagram
- Timing Diagram

Structural Diagrams (Static View):

A- Class Diagram

What is a Class Diagram?

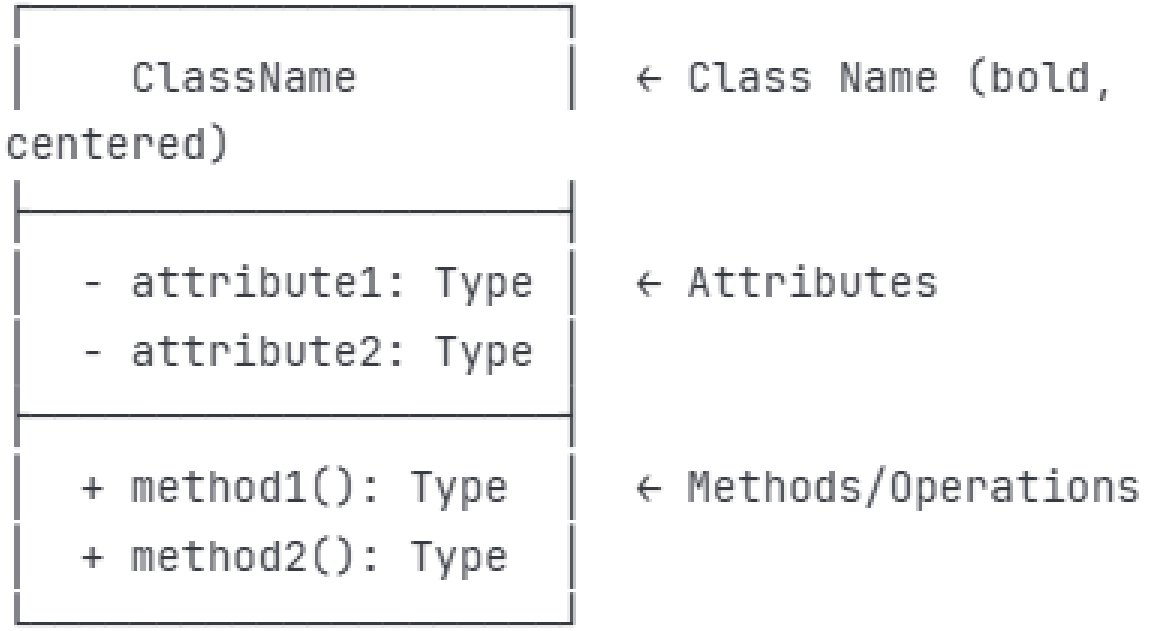
- Most commonly used UML diagram
- Shows the static structure of a system
- **Displays classes, attributes, methods, and relationships**
- Provides a blueprint for implementation

Uses:

- Database schema design.

Structural Diagrams (Static View):

Basic Class Structure:



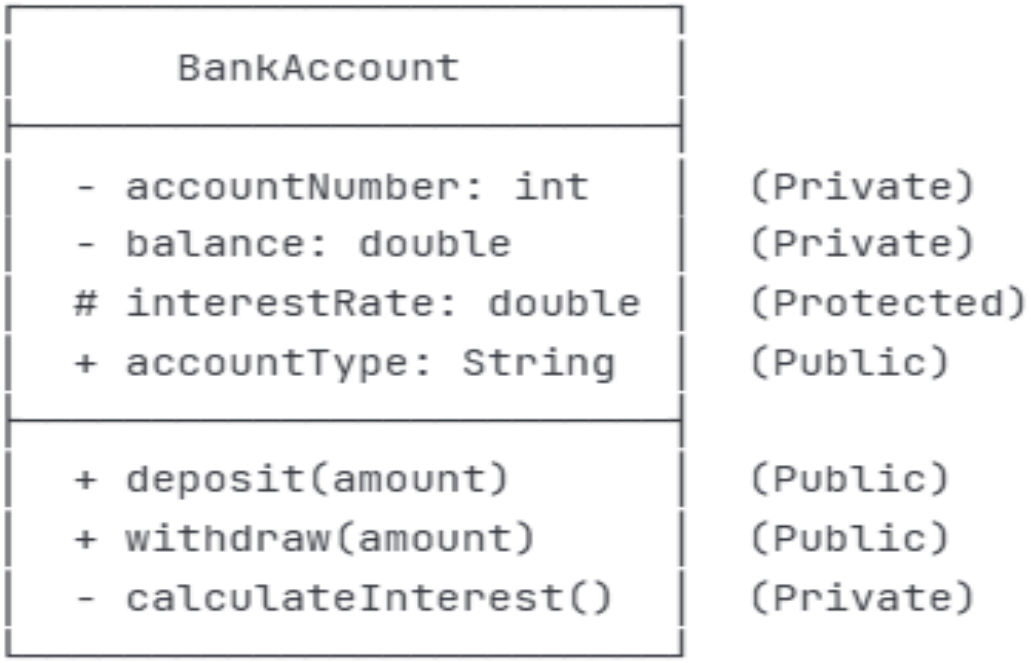
- **Three Compartments:**
- **Name Compartment**: Class name
- **Attribute Compartment**: Properties/fields
- **Operation Compartment**: Methods/functions

Structural Diagrams (Static View):

Visibility Modifiers

- **+ Public**: Accessible from anywhere
- **- Private**: Accessible only within the class
- **# Protected**: Accessible within class and subclasses
- **~ Package**: Accessible within the same package

Example:



Attributes

- **Attribute Syntax:**

visibility name: type [multiplicity] = defaultValue {properties}

- **Examples:**

- name: String

+ age: int = 0

grades: double[*] (array/collection)

- isActive: boolean = true

Attributes

Derived Attributes:

- Calculated from other attributes
- Notation: /derivedAttribute
- Example: /age: int (calculated from birthdate)

Static Attributes:

- Belong to the class, not instances
- Notation: underlined
- Example: count: int (underlined)

Methods/Operations

- **Method Syntax:**

visibility name(parameters): returnType {properties}

Examples:

+ getName(): String

+ setAge(age: int): void

- calculateTotal(price: double, quantity: int): double

validateInput(data: String): boolean

Methods/Operations

- **Method Syntax:**

visibility name(parameters): returnType {properties}

Examples:

+ getName(): String

+ setAge(age: int): void

- calculateTotal(price: double, quantity: int): double

validateInput(data: String): boolean

Methods/Operations

- **Static Methods:**

  - Belong to the class, not instances
  - Notation: underlined
  - Example: getInstance(): Singleton (underlined)

- **Abstract Methods:**

  - Must be implemented by subclasses
  - Notation: italicized
  - Example: draw(): void (italicized)

Relationships in Class Diagrams

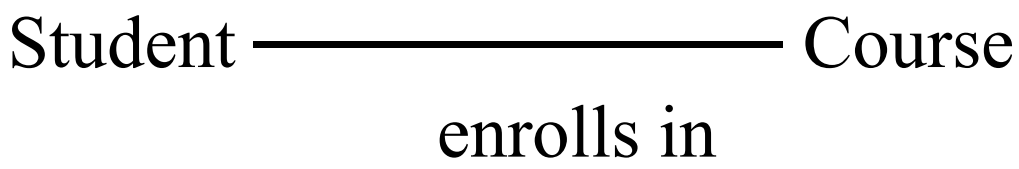
- **Main Types of Relationships:**
  - **Association** - "uses-a" or "has-a"
  - **Aggregation** - "has-a" (weak ownership)
  - **Composition** - "contains-a" (strong ownership)
  - **Inheritance/Generalization** - "is-a"

•**Association:**

•Represents a relationship between two classes
•Notation: Solid line connecting classes
•Can be unidirectional or bidirectional

Relationships in Class Diagrams

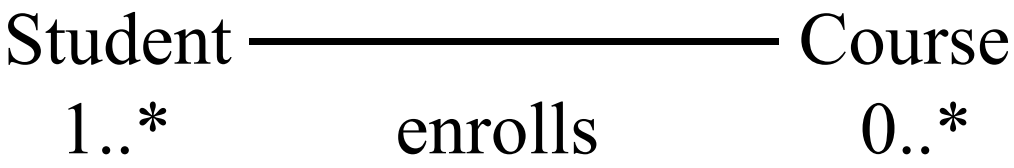
- Example:

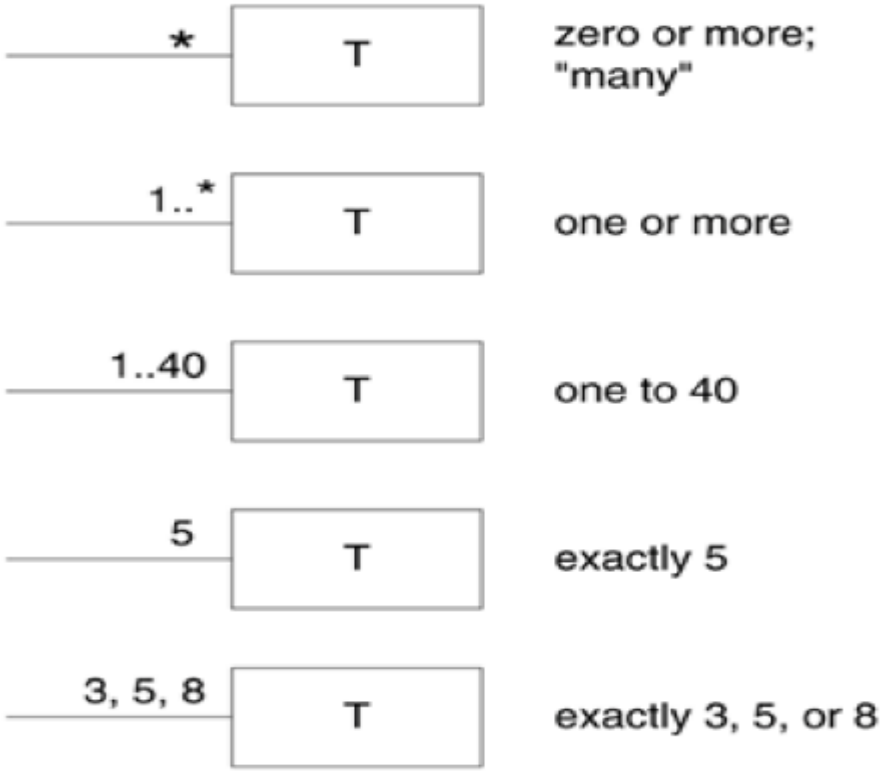


- **With Multiplicity:**

Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class.

- Example:





•Aggregation

- Special type of association (whole-part relationship)
- "Has-a" relationship with weak ownership
- Parts can exist independently of the whole
- Notation: Hollow diamond on the "whole" side

❑ Example: Department ◇———— Employee

contains

Relationships in Class Diagrams

- **Real-world Examples:**
- University has Departments
- Team has Players
- Library has Books
- Company has Employees

Parts can exist without the whole (Employee can exist without Department)

Relationships in Class Diagrams

- **Composition :**

- Stronger form of aggregation
- "Contains-a" relationship with strong ownership
- Parts cannot exist independently of the whole
- When whole is destroyed, parts are destroyed
- Notation: Filled diamond on the "whole" side

- Example:

Relationships in Class Diagrams

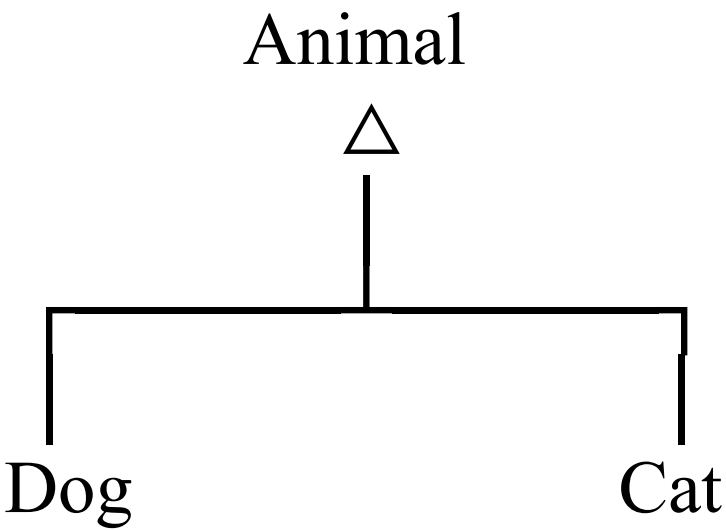
- **Real-world Examples:**
- House has Rooms
- Car has Engine
- Book has Pages
- Human has Heart

Parts cannot exist without the whole (Room cannot exist without House)

Relationships in Class Diagrams

- **Inheritance/Generalization**
- "Is-a" relationship
- Subclass inherits attributes and methods from superclass
- Notation: Hollow triangle arrow pointing to parent class

- Example:



Relationships in Class Diagrams

Example Hierarchy:

- Vehicle → Car, Truck, Motorcycle
- Shape → Circle, Rectangle, Triangle
- Employee → Manager, Developer, Designer

Example 1: University Management System

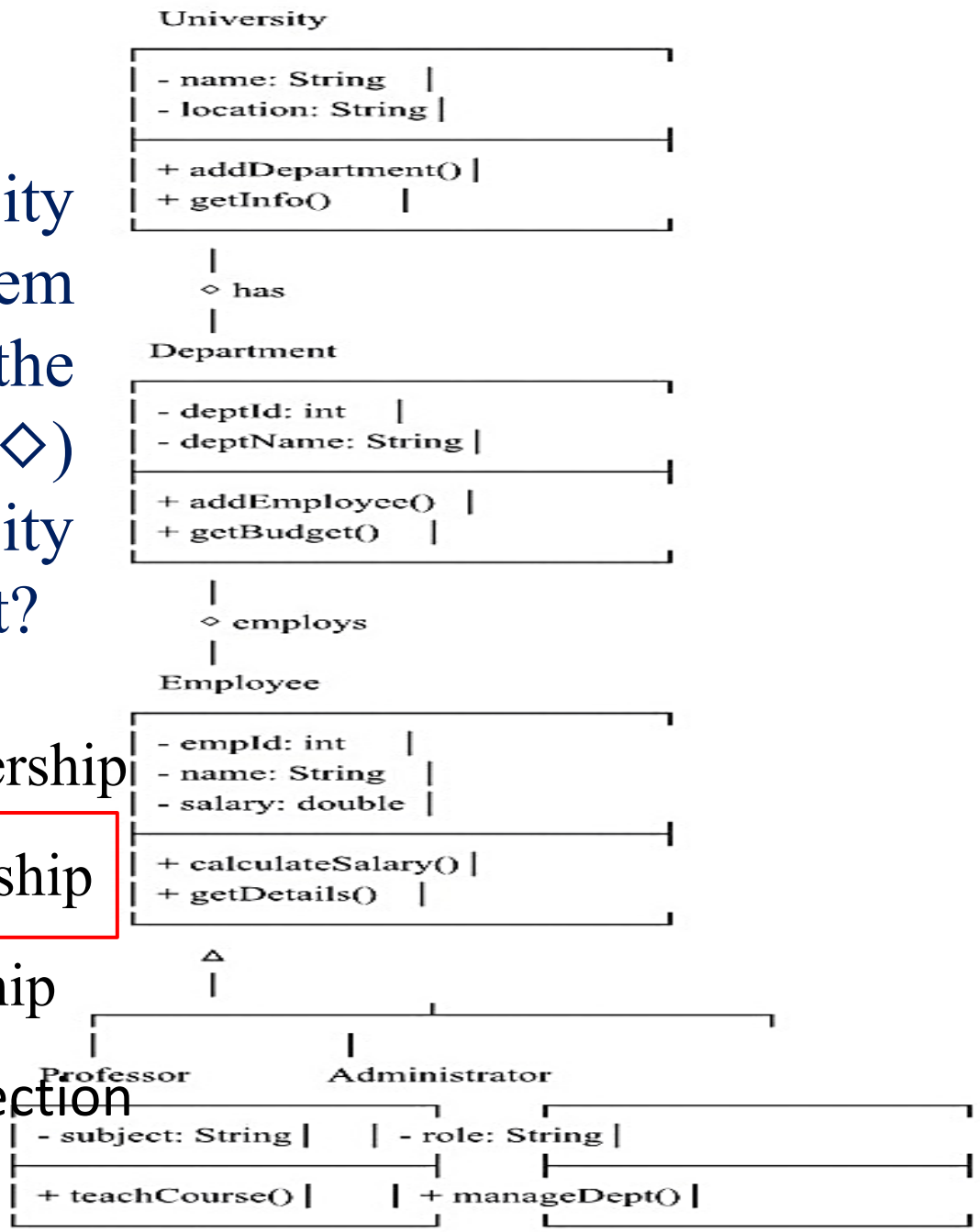
1- In the University Management System diagram, what does the hollow diamond (◇) symbol between University and Department represent?

a) Composition - strong ownership

b) Aggregation - weak ownership

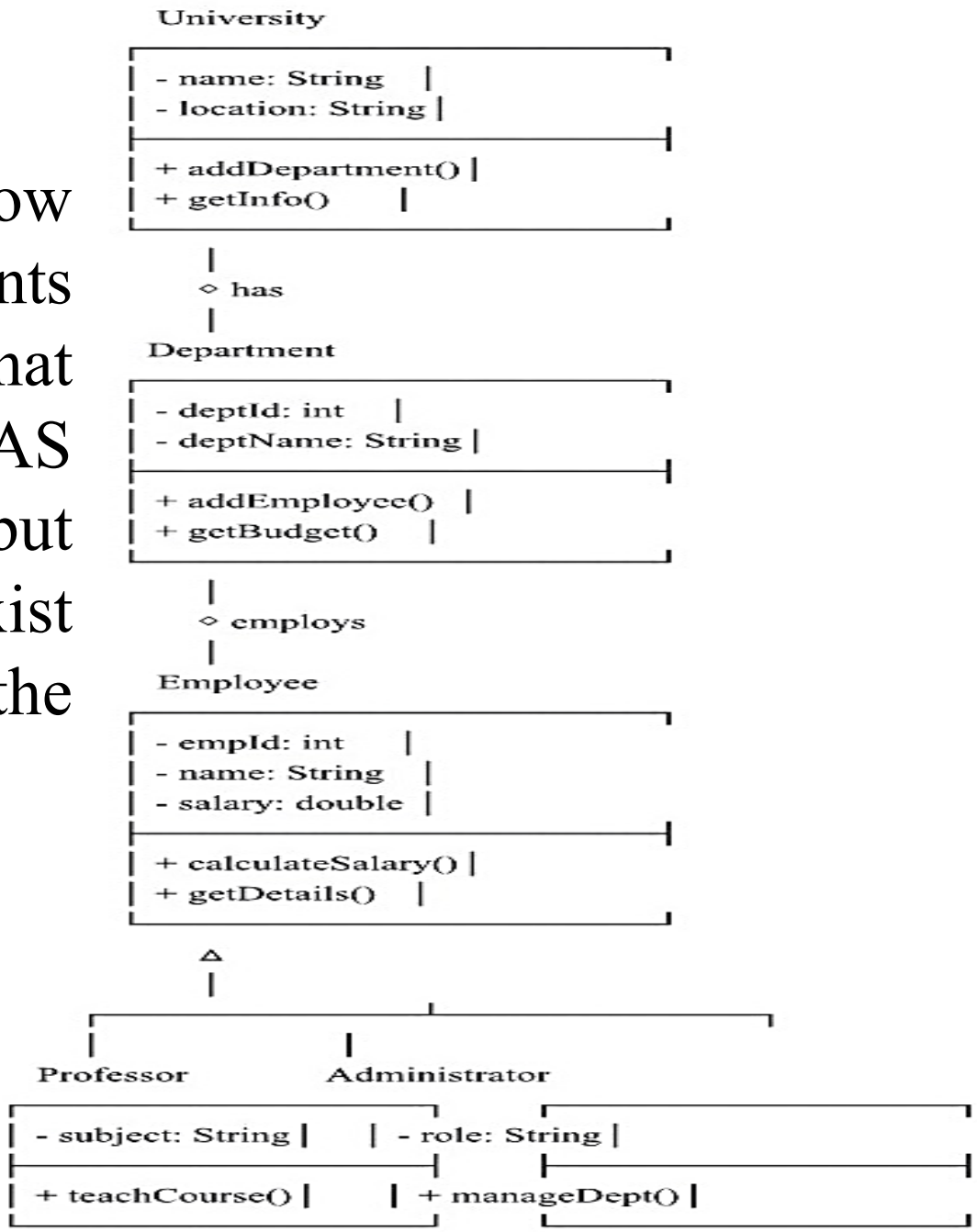
c) Inheritance - is-a relationship

d) Association - simple connection



Example 1: University Management System

Explanation: The hollow diamond (◇) represents aggregation, indicating that University HAS Departments, but departments can exist independently of the university.



Example 1: University Management System

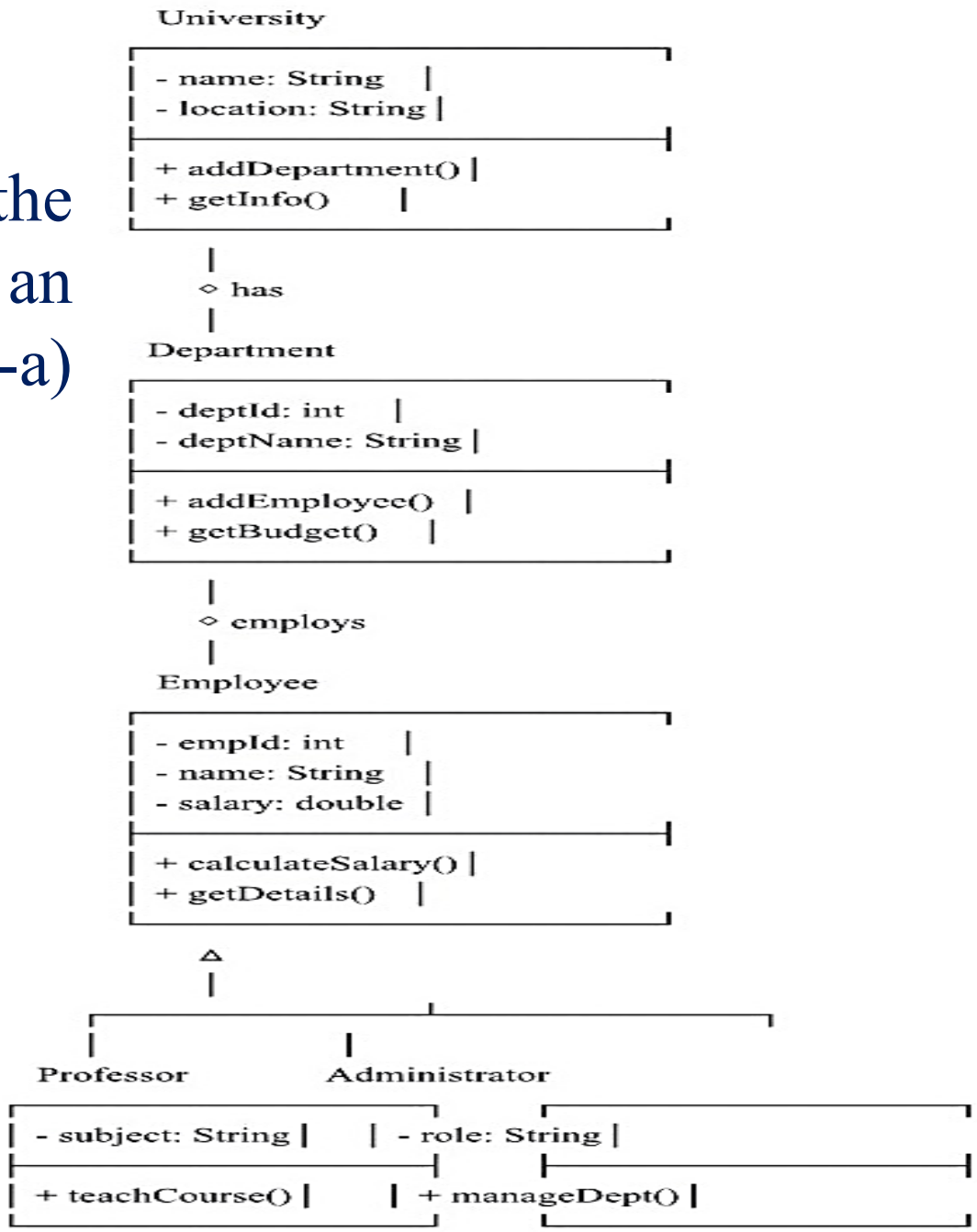
2- Which symbol in the diagrams represents an inheritance (is-a) relationship?

a) Filled diamond (◆)

b) Hollow diamond (◇)

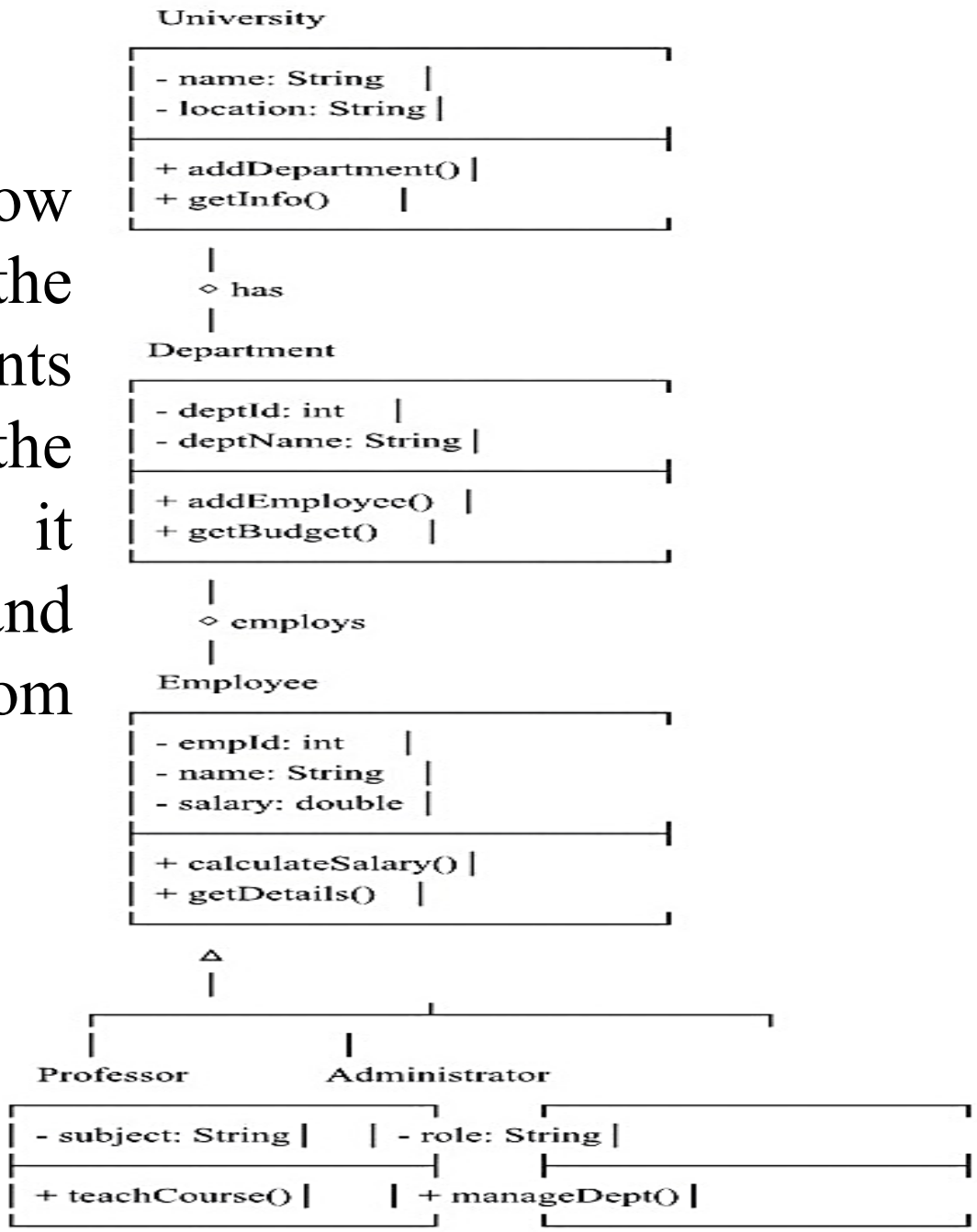
c) Hollow triangle (△)

d) Solid line (——)



Example 1: University Management System

Explanation: The hollow triangle pointing to the parent class represents inheritance. In the University diagram, it shows that Professor and Administrator inherit from Employee.



Example 1: University Management System

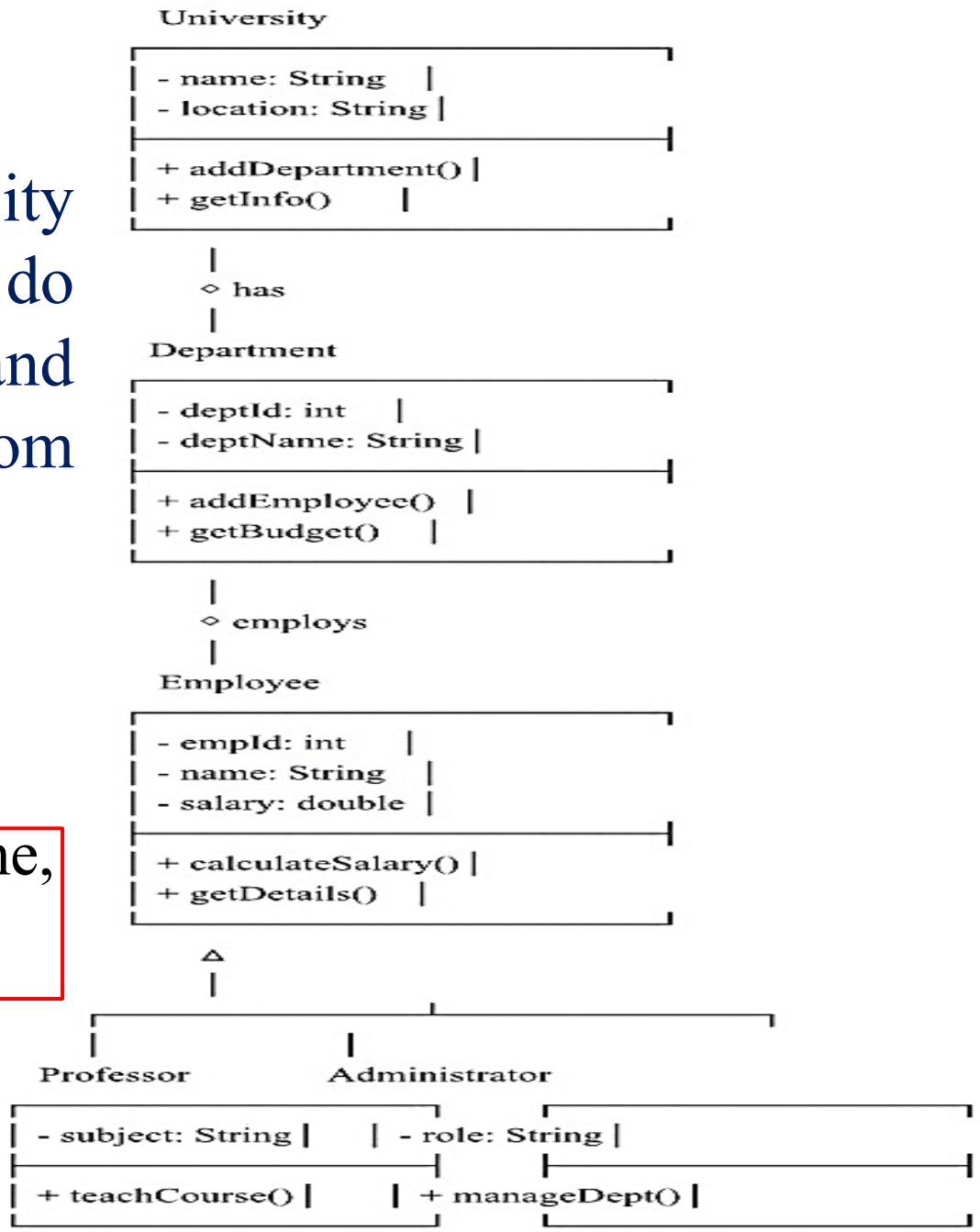
3- In the University System, what attributes do Professor and Administrator inherit from Employee?

a) Only empId

b) Only name and salary

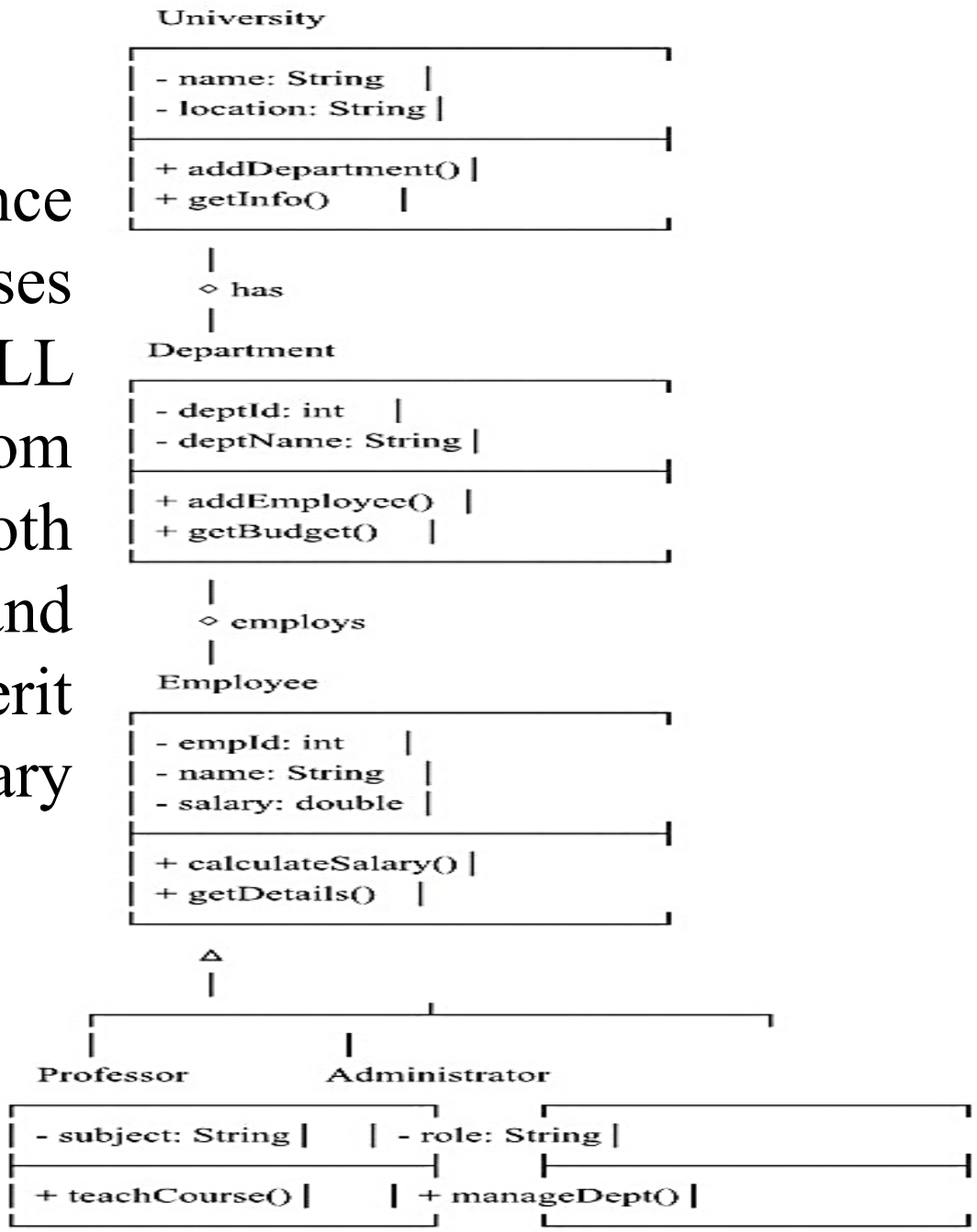
c) All attributes: empId, name, and salary

d) They don't inherit any attributes



Example 1: University Management System

Explanation: Inheritance means subclasses automatically get ALL attributes and methods from the parent class. Both Professor and Administrator inherit empId, name, and salary from Employee.



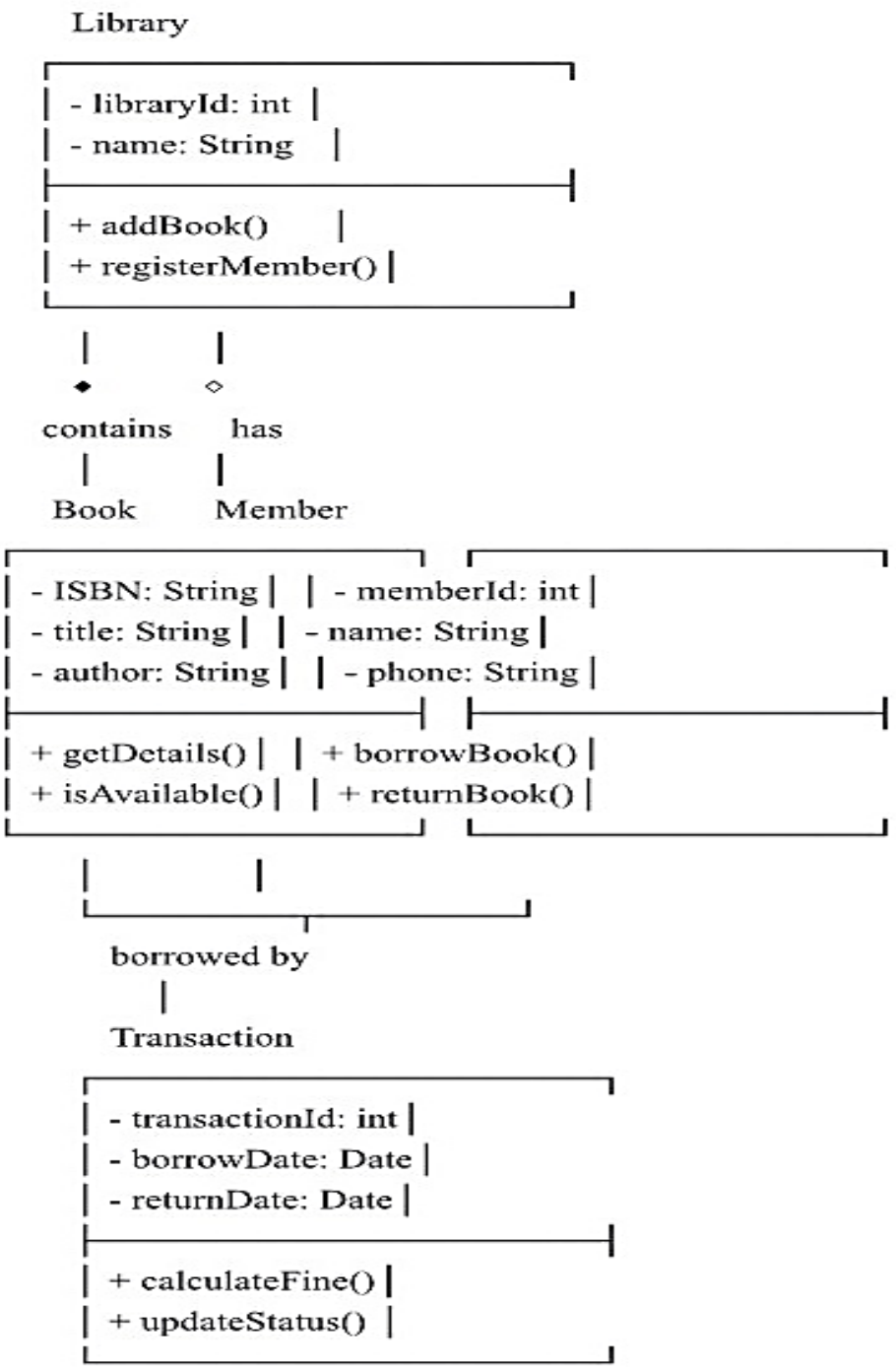
Example 2: Library Management System

1- What is the main difference between Library-Book (◆) and Library-Member (◇) relationships?

a) Both are the same type of relationship

b) Book uses composition (strong ownership); Member uses aggrega (weak ownership)

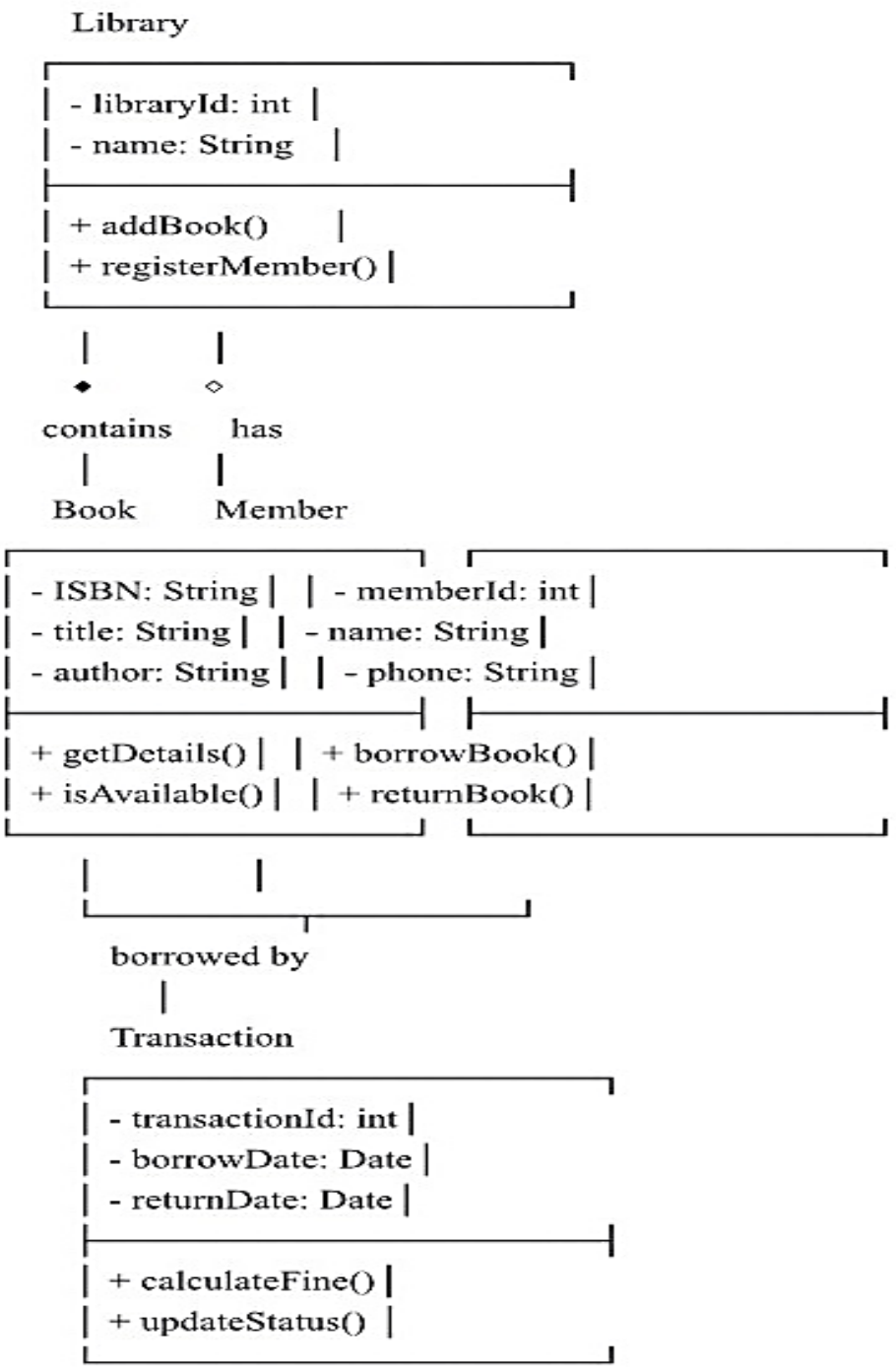
c) Book uses aggregation; Membe uses composition



Example 2: Library Management System

Explanation:

- Filled diamond (◆) = Composition: Books are destroyed when Library is destroyed
- Hollow diamond (◇) = Aggregation: Members continue to exist independently



Example 2: Library Management System

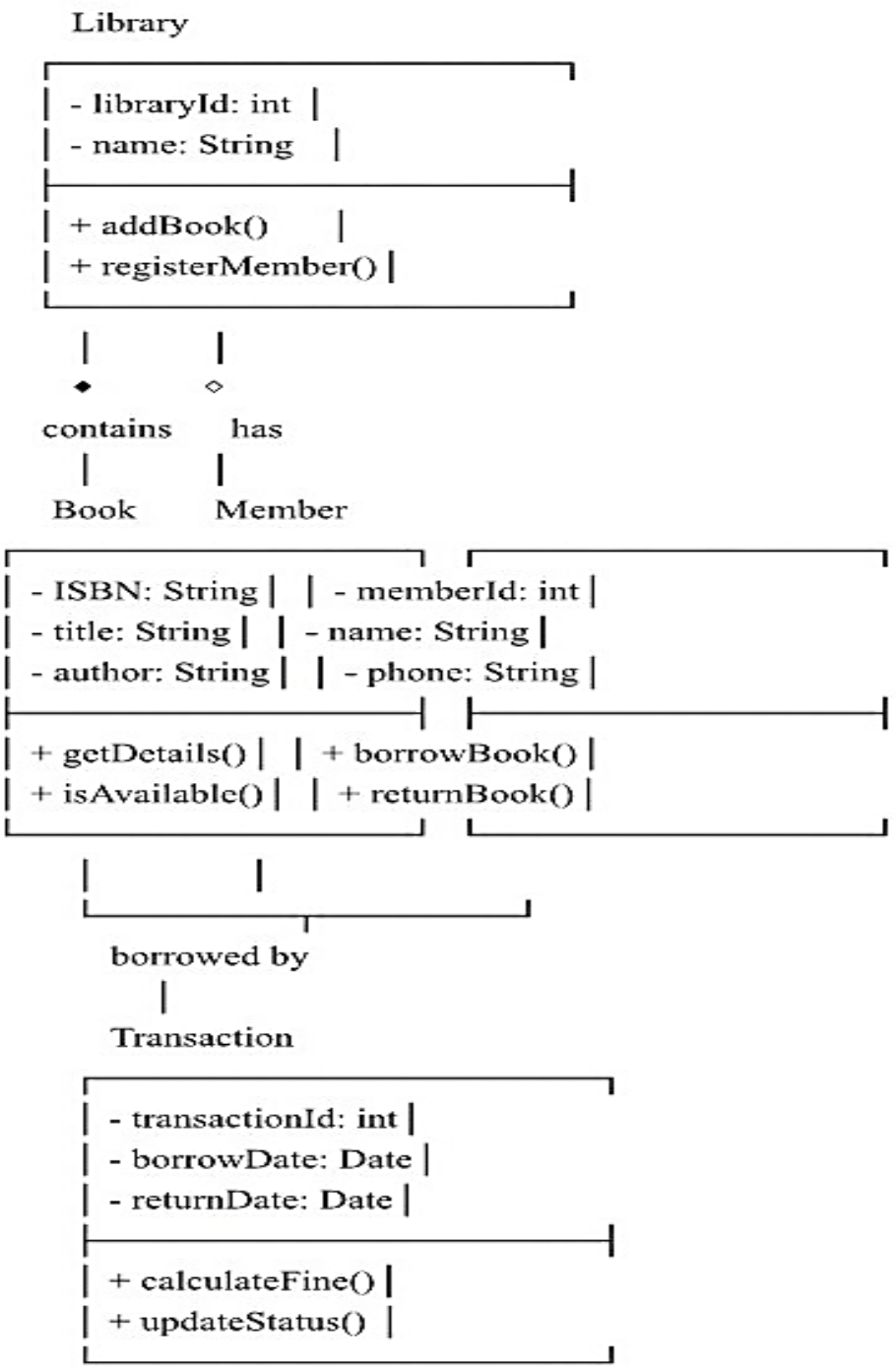
2- In the Book class, what does "- ISBN: String" indicate about the visibility of the ISBN attribute?

a) Public - accessible everywhere

b) Private - accessible only within Book class

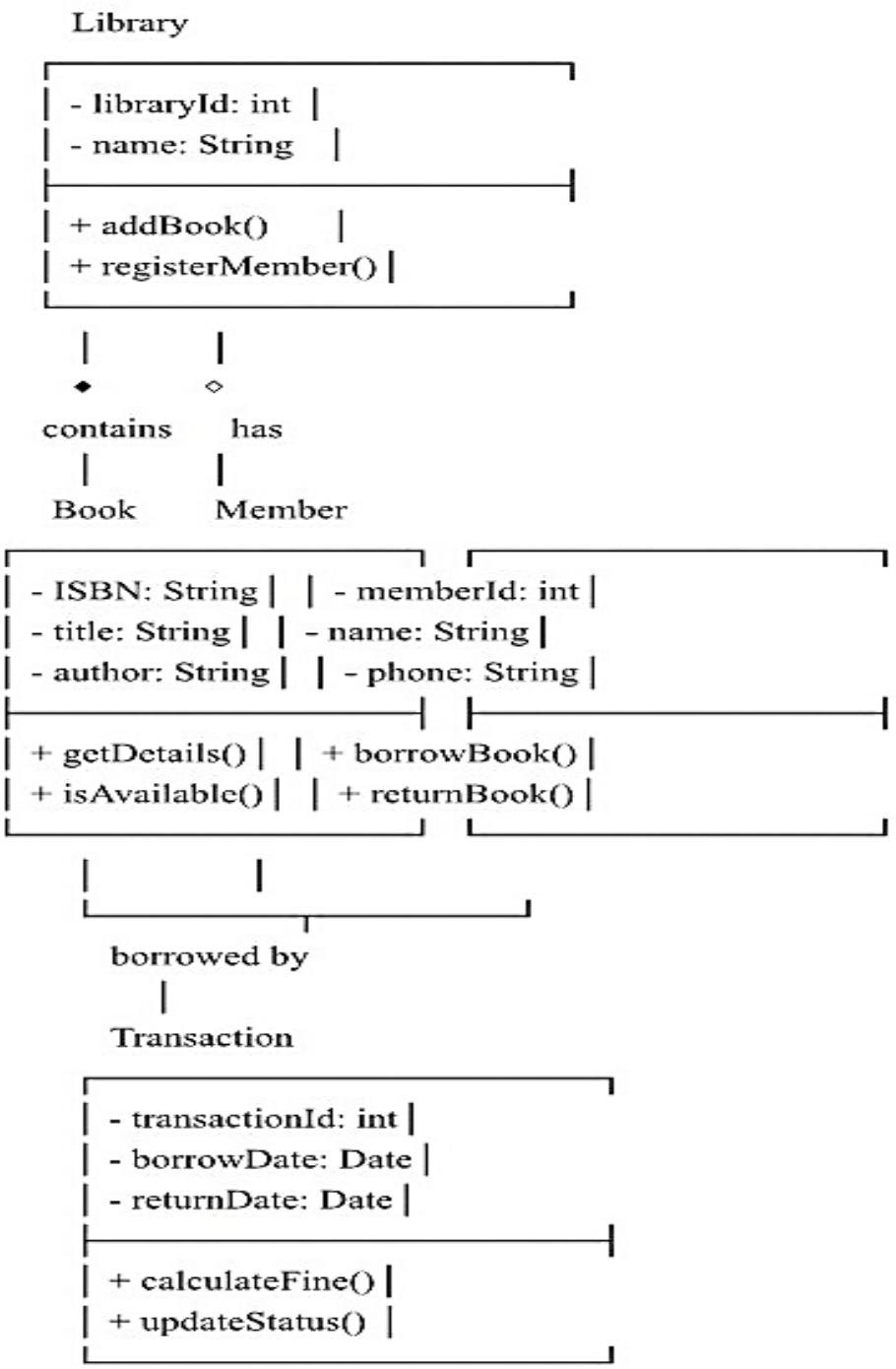
c) Protected - accessible in Book a subclasses

d) Package - accessible in same package



Example 2: Library Management System

Explanation: The minus sign (-) represents private visibility, meaning ISBN can only be accessed within the Book class



EX (3)

- **A complete UML class box consists of how many compartments?**

a) 1 compartment (name only)

b) 2 compartments (name and attributes)

c) 3 compartments (name, attributes, and methods)

d) 4 compartments (name, attributes, methods, and relationships).

EX (4)

- **Which statement is TRUE about composition relationships?**

a) Parts can exist independently of the whole

b) Parts are destroyed when the whole is destroyed

c) It's the same as aggregation

d) It uses a hollow diamond symbol

EX (5)

- **What does the method notation "+ borrowBook()" indicate??**

a) borrowBook is a private method

b) borrowBook is a public method with no parameters and no return value

c) borrowBook is a protected attribute

d) borrowBook is a static method